

Camera Calibration and Perspective-Based Real-World Measurement

CSc 8830 - Computer Vision, Module 2 Assignment

Name: _____
Course Section: _____
Date: _____

GitHub repository: https://github.com/soonebabu/CV_CameraCalibration
(replace with your repo URL if this changes before submission)

1. Overview

This assignment has three working parts and one theory part. The working parts build on each other: first the phone camera is calibrated in OpenCV to recover its intrinsic parameters, then those parameters are used to convert a pixel measurement in a photo into a real-world length via the perspective projection equation, and finally that measurement approach is validated against twenty independently measured objects to see how accurate it actually is in practice. The theory part is separate - it asks for the mathematical relationship between two cameras looking at the same 3D point, worked out from first principles rather than run as code.

All code referenced here lives in the GitHub repository linked above, each script commented and with a usage note at the top of the file. The same pipeline is also wrapped in a small Flask web app (`app.py`) so it can be run and demonstrated from a browser rather than a terminal, per the assignment's web-demo requirement.

2. Step 1 - Camera Calibration

Calibration was performed with OpenCV's standard chessboard method (`cv2.findChessboardCorners / cv2.calibrateCamera`), implemented in `calibration.py`. A printed chessboard (9x6 inner corners, 25 mm squares) was photographed with a smartphone camera from 15-25 different positions and angles, keeping the zoom level fixed across every shot so the intrinsics stay valid for later steps. Corner locations were refined to sub-pixel accuracy with `cv2.cornerSubPix` before being passed to the solver, which noticeably tightens the reprojection error compared to using the raw integer corner detections.

What calibration recovers

OpenCV's calibration returns the 3x3 camera (intrinsic) matrix K and a 5-element distortion vector:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

f_x and f_y are the focal length expressed in pixel units along each image axis (they already fold in the sensor's physical pixel pitch, which is why Step 2 can use them directly without ever needing the actual millimeter focal length or sensor size printed in a phone's spec sheet). c_x and c_y are the principal point - where the optical axis actually pierces the image plane, which is close to the image center but rarely exactly on it. The distortion vector (k_1, k_2, p_1, p_2, k_3) captures the phone lens's radial and tangential distortion, which is corrected out of the clicked pixel points in Step 2 before the flat pinhole formula is applied.

Fill in with your own results: run `python calibration.py --images calibration_images --cols 9 --rows 6 --square 25.0` on your own chessboard photos and paste the printed f_x , f_y , c_x , c_y , distortion coefficients, and mean reprojection error here. A reprojection error under roughly 1 pixel is a reasonable target for a phone camera; if it comes out much higher, drop whichever image the script flags as the worst offender and re-run.

3. Step 2 - Real-World Dimensions from Perspective Projection

The pinhole camera model says a 3D point at distance Z from the camera, displaced by a real-world length L perpendicular to the optical axis, lands on the image sensor displaced by a length that depends on the focal length and shrinks with distance - exactly the similar-triangles relationship a simple lens produces:

$$p / f = L / Z$$

where p is the length in pixels, f is the calibrated focal length in pixels (f_x or f_y , whichever axis matches the measurement direction), L is the real-world length, and Z is the camera-to-object distance.

Rearranged for what we actually want:

$$L = (p \times Z) / f$$

`measure_dimensions.py` implements exactly this. Given a photo, the user clicks the two endpoints of the edge to be measured; those pixel coordinates are first undistorted using the distortion coefficients from Step 1 (`cv2.undistortPoints`), then split into horizontal and vertical pixel displacements, each converted to millimeters using f_x and f_y respectively, and recombined with the Pythagorean theorem to get the final length. Using f_x/f_y separately rather than a single averaged focal length keeps the result exact even when a phone sensor's horizontal and vertical pixel pitch aren't perfectly identical.

Assumptions this relies on

- The measured edge lies on a plane that is roughly perpendicular to the camera's optical axis (fronto-parallel). Tilting the object relative to the camera introduces foreshortening that this single linear formula doesn't correct for.
- Z is the distance from the camera's optical center to that plane, not to the phone's back casing - for distances beyond a couple of meters this distinction is well within measurement noise anyway.
- Pixel coordinates are undistorted before the formula is applied, so the remaining relationship is the ideal linear pinhole model even though a real phone lens is not perfectly rectilinear.

4. Step 3 - Validation Over 20 Objects

To check how well Step 2 holds up in practice, twenty objects were photographed from a fixed distance greater than 2 meters, with the real-world length of each measured independently by tape measure as ground truth. `validate_measurements.py` takes a CSV of (actual length, measured length) pairs and reports the mean absolute error, RMSE, mean percentage error, and standard deviation of the error, along with two comparison plots.

Software validation, not the physical experiment: the table and plots below come from `demo_data/demo_validation.csv`, a synthetically generated dataset used only to prove the statistics script and plots work correctly end to end. They are not a substitute for the real 20-object experiment the assignment requires - replace this file with your own actual/measured pairs (see `sample_measurements.csv` in the repo for the expected format) and re-run `validate_measurements.py` before submitting, then swap this section for your own numbers and plots.

Object	Actual (mm)	Measured (mm)	Error (mm)	% Error
obj_01	222.5	216.6	-5.9	2.7%
obj_02	293.3	292.5	-0.8	0.3%
obj_03	261.7	257.6	-4.1	1.6%
obj_04	118.6	119.5	0.9	0.8%
obj_05	138.0	138.5	0.5	0.4%
obj_06	287.1	286.5	-0.6	0.2%
obj_07	61.4	53.3	-8.1	13.2%
obj_08	273.5	271.8	-1.7	0.6%
obj_09	267.2	267.0	-0.2	0.1%
obj_10	181.7	182.1	0.4	0.2%
obj_11	138.8	133.9	-4.9	3.5%
obj_12	132.4	130.9	-1.5	1.1%
obj_13	126.3	123.2	-3.1	2.5%
obj_14	175.7	173.1	-2.6	1.5%
obj_15	191.2	194.6	3.4	1.8%
obj_16	203.9	201.3	-2.6	1.3%
obj_17	318.8	318.7	-0.1	0.0%
obj_18	266.1	268.9	2.8	1.1%
obj_19	221.8	219.9	-1.9	0.9%
obj_20	317.1	316.7	-0.4	0.1%

Summary statistics (demo run):

=====

n objects: 20

mean error mm: -1.525

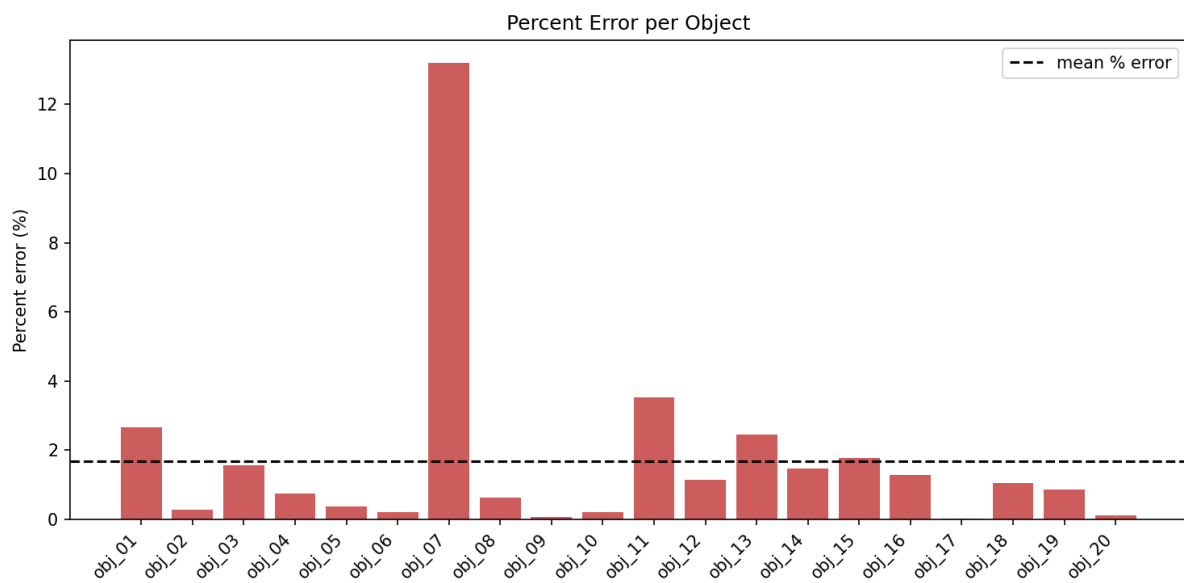
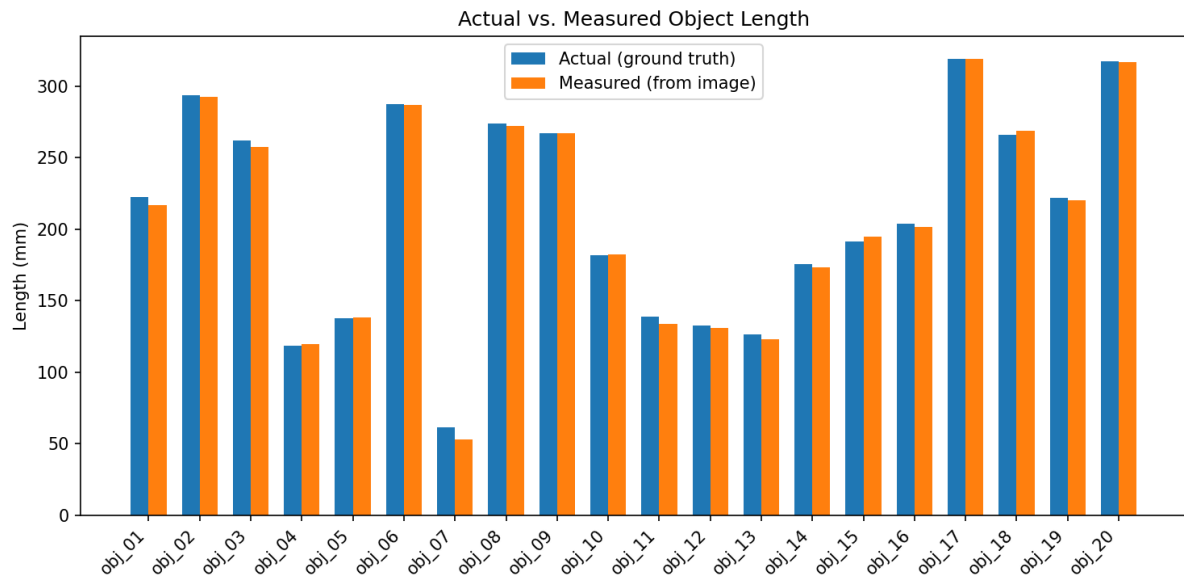
mean abs error mm: 2.325

std error mm: 2.795

rmse mm: 3.122

mean pct error: 1.682

Plots (demo run)



5. Theory - Relating Two Cameras Viewing the Same Point

Problem: camera 1 is static. Camera 2 sits at some distance from camera 1, at an oblique orientation - i.e. rotated relative to camera 1, not just shifted sideways. A 3D point P is visible to both. Derive the relationship between the image coordinates of P in camera 1 and in camera 2.

5.1 Setup and notation

Take camera 1's own coordinate frame as the world frame - a free choice, justified simply because camera 1 is the one that isn't moving, so anchoring the world to it means camera 1's projection equation stays as simple as possible. Let P have coordinates $X_1 = (X_1, Y_1, Z_1)$ in this frame.

Camera 2's pose relative to camera 1 is described by a rotation matrix R (3x3, orthonormal, $\det R = 1$) and a translation vector t (3x1). These are the two static extrinsic parameters that capture camera 2's oblique orientation and offset. A point given in camera 1's frame converts to camera 2's frame by the standard rigid-body transform:

$$X_2 = R X_1 + t$$

Each camera has its own intrinsic matrix, K_1 and K_2 (same form as in Section 2), obtained by running the Step 1 calibration procedure separately on each camera. Camera 1's projection is then the simple case of the general pinhole equation (identity rotation, zero translation, since the world frame is camera 1's frame):

$$Z_1 p_1 = K_1 X_1 \quad (p_1 = (u_1, v_1, 1)^T \text{ in homogeneous pixel coordinates})$$

and camera 2's projection is:

$$Z_2 p_2 = K_2 X_2 = K_2 (R X_1 + t)$$

5.2 Direct point transfer (when depth is known)

If the depth Z_1 of P from camera 1 happens to be known - e.g. it was measured directly, or triangulated from an earlier pair of views - then the exact corresponding pixel in camera 2 can be computed outright. Back out the 3D point from camera 1's pixel coordinates:

$$X_1 = Z_1 K_1^{-1} p_1$$

transform it into camera 2's frame, and reproject:

$$X_2 = R X_1 + t \quad p_2 = (1/Z_2) K_2 X_2 \quad (Z_2 = \text{third component of } X_2)$$

This is a complete, depth-dependent answer, but it needs Z_1 as an extra piece of information that a single image alone doesn't give you. The more generally useful result - and the one usually meant by "the relationship between two cameras" - is the constraint below, which holds for the corresponding pixels regardless of depth.

5.3 The depth-independent constraint (epipolar geometry)

Define normalized image rays (undoing the intrinsics, so these are pure directions from each camera's optical center through the image point, expressed in that camera's own frame):

$$x1 = K1^{-1} p1 \quad x2 = K2^{-1} p2$$

Camera 2's optical center, expressed back in camera 1's frame, is found by setting $X2 = 0$ in the rigid transform: $0 = R \cdot C + t$, so $C = -R^T t$. The three points camera 1's center (the origin), camera 2's center C , and the 3D point P all lie in one plane - the epipolar plane - simply because any three points in space define a plane. Equivalently, the three vectors drawn from camera 1's center are coplanar: the ray to P (direction $x1$), the baseline to camera 2's center (direction $C = -R^T t$), and the ray to P as seen from camera 2 but rotated back into camera 1's frame (direction $R^T x2$). Coplanar vectors have a zero scalar triple product:

$$x1 \cdot [(-R^T t) \times (R^T x2)] = 0$$

A rotation applied to both operands of a cross product just rotates the result (since $\det R = 1$), so $(R^T a) \times (R^T b) = R^T(a \times b)$. That simplifies the bracket to $-R^T(t \times x2)$, giving:

$$-x1^T R^T (t \times x2) = 0 \quad \rightarrow \quad x2^T [t]_{\times} R x1 = 0$$

where $[t]_{\times}$ is the 3x3 skew-symmetric matrix built from t so that $[t]_{\times} v = t \times v$ for any vector v (the cross product with t rewritten as a matrix multiply). Defining the Essential matrix $E = [t]_{\times} R$ gives the compact epipolar constraint:

$$x2^T E x1 = 0 \quad E = [t]_{\times} R$$

and, substituting back $x1 = K1^{-1}p1$, $x2 = K2^{-1}p2$, the same relationship directly in pixel coordinates, using the Fundamental matrix F :

$$p2^T F p1 = 0 \quad F = K2^{-T} [t]_{\times} R K1^{-1}$$

This is the answer to the question as posed: it relates the image coordinates $p1$ and $p2$ of the same 3D point across the two cameras without needing to know how far away that point actually is.

Geometrically, for a given pixel $p1$ in camera 1, the equation $p2^T F p1 = 0$ restricts where its match $p2$ can be in camera 2 to a single line (the epipolar line) - not a random search over the whole image.

5.4 Static parameters, variables, and how each is obtained

Parameter	Meaning	How it's determined
K1, K2	Intrinsics of each camera (fx, fy, cx, cy)	Step 1 chessboard calibration, run once per camera
Distortion coeffs (per camera)	Radial/tangential lens distortion	Also from Step 1 calibration; used to undistort $p1/p2$ before applying the linear equations above
R	Camera 2's orientation relative to camera 1	Stereo calibration (cv2.stereoCalibrate) if both cameras see a shared calibration target at the same time; otherwise estimated from point correspondences via the Essential matrix (cv2.findEssentialMat + cv2.recoverPose)
t	Camera 2's position relative to camera 1 (baseline)	Same stereo calibration step; note that recovering E from point correspondences alone only gives t up to an unknown scale, so an independently known length in the scene (or a physically measured baseline) is needed to fix real-world units

Variable	Meaning	How it's determined
$p1 = (u1, v1)$	Pixel location of P in camera 1	Directly observed / detected in image 1

$p_2 = (u_2, v_2)$	Pixel location of P in camera 2	Directly observed / detected in image 2 (found via matching, or constrained to the epipolar line from p_1)
Z_1, Z_2	Depth of P along each camera's optical axis	Not needed for the epipolar constraint itself; if actually wanted, recovered by triangulation (<code>cv2.triangulatePoints</code>) once K_1, K_2, R, t and a correspondence (p_1, p_2) are known

5.5 Assumptions made, and why they're reasonable here

- **P is a fixed, rigid point in the scene, observed either simultaneously by both cameras or while it hasn't moved between the two shots.** Without this, "the same point P" in both images wouldn't refer to the same physical location, and the whole correspondence breaks down.
- **Both cameras are well-modeled as pinhole cameras after distortion correction.** Real lenses distort straight lines, but Step 1's calibration already gives per-camera distortion coefficients, so undistorting p_1 and p_2 first makes the linear equations above legitimate.
- **K_1, K_2, R and t are constant for the duration of capture** - i.e. neither camera changes focus/zoom, and camera 2 doesn't move, between calibration and the actual observation of P. If camera 2 can move freely between shots, R and t would need to be re-estimated each time rather than treated as fixed extrinsics.
- **P actually falls inside the overlapping field of view of both cameras**, which is given in the problem statement and is also required for any correspondence to exist in the first place.
- **The baseline t is non-degenerate** (camera 2 isn't sitting essentially on the same optical center as camera 1, and isn't lined up so that the ray to P is parallel to the baseline). This isn't needed for the epipolar equation to be algebraically valid, but a very small or degenerate baseline makes depth recovery from triangulation numerically unstable in practice, which matters if Z is ever needed.

6. Conclusion

The three practical steps form one continuous pipeline: calibration turns a folder of ordinary photos into a precise geometric model of the camera (K , distortion), that model turns a single clicked measurement plus a known distance into a real-world length, and the validation step gives an honest read on how much error that whole chain actually carries once it's tested against a tape measure over a reasonable number of objects. The theory section extends the same pinhole model to two cameras and shows that even without knowing depth, the geometry alone pins down where a matching point must lie in the second image - the basis for stereo vision and multi-camera 3D reconstruction generally.

Code, README, and the web app referenced throughout this document are in the GitHub repository linked on the title page.